NEXUS ANALYTICS

Architecture & Engineering Note
Prototype architecture for the Npontu Technologies practical assignment

1. Architecture overview

Nexus Analytics uses a modular monolithic architecture. The Next.js frontend communicates with a FastAPI backend through a same-origin rewrite. The backend authenticates business users, validates and ingests datasets, runs parameterised SQL analytics, loads a saved Scikit-learn model, calls Snwolley server-to-server and renders PDF reports.

Business user	Next.js frontend	FastAPI backend
Registration and decisions	Workspace, data and charts	Authentication and orchestration

Backend services: PostgreSQL persistence · Scikit-learn training/inference · Snwolley Agent API · ReportLab PDF renderer

Layer	Responsibility	Technology
Presentation	Onboarding, workspace, data, charts and reports	Next.js / React / Recharts
Application API	Authentication, validation, orchestration and error handling	FastAPI / Pydantic
Persistence	Business, membership, workspace, dataset and transaction data	PostgreSQL 16 / psycopg
ML	Preprocessing, training, evaluation and inference	Scikit-learn / Joblib
AI	Evidence-bounded insight and management narrative	Snwolley Agent API
Reporting	Business-facing PDF generation	ReportLab

2. Business and tenant isolation

Registration creates a user, business, owner membership, default workspace and analytical objective in a single database transaction. Each authenticated request resolves the caller's business from a signed session cookie. A caller-supplied business identifier is accepted only if it matches the authenticated membership. Customers, agents and transactions use composite primary keys containing business_id; composite foreign keys prevent cross-business customer/agent references.

3. Security controls

Passwords use PBKDF2-HMAC-SHA256 with random salts. Sessions are HMAC-signed, expire after 12 hours and are stored in HTTP-only SameSite=Lax cookies. Secure cookies are enabled when APP_ENV=production. Database statements use bound parameters. Snwolley credentials are stored server-side in environment configuration. AI prompts contain aggregates and selected model metrics rather than individual customer records. Provider HTTP 429 responses preserve Retry-After.

4. Data and ML workflow

The data generator uses a fixed seed and creates 30,000 synthetic transaction records. The loader creates or locates a demo business and only replaces that tenant's synthetic transaction rows. CSV ingestion validates schema, duplicate IDs, amount and timestamp validity, and uses PostgreSQL COPY for bulk loading. SQL demonstrates joins, filters, aggregation, conditional aggregation, CTEs and window functions.

The model pipeline imputes missing numeric/categorical fields, scales numeric variables, one-hot encodes categories and fits a class-weighted Random Forest. A DummyClassifier is the baseline. Holdout evaluation reports ROC AUC, average precision, precision, recall and F1. The prediction endpoint returns a risk score and review recommendation; it does not declare fraud.

5. AI augmentation and reporting

The Snwolley Agent receives the business question plus transaction aggregates, type/channel summaries and model metrics. Prompt instructions require evidence-bounded responses, separation of observation and hypothesis, no unsupported causal claims, and explicit synthetic-data caveats. The UI presents the narrative as advisory management analysis. A separate endpoint builds a PDF containing observed metrics and the AI narrative. The provider's stated limit is five requests per minute; the current implementation surfaces 429 and Retry-After but does not use a distributed queue.

6. Scaling and production evolution

For larger datasets, ingestion could move to scheduled pipelines or Kafka-backed streaming. Data quality and feature engineering could run in background workers. Model registry/versioning and asynchronous training would support controlled deployment. Elasticsearch could support search and log analysis; Prometheus/Grafana could monitor latency, database health, ingestion lag, model drift and provider errors. These are proposed extension points, not currently deployed services.

7. Known limitations

The current model is limited to the fraud-risk demonstration; other selected objectives are recorded but need their own feature mappings, training pipelines and inference contracts. Model precision is low, scores are not calibrated and labels are synthetic. Password reset, email verification, login throttling, granular RBAC, invitation management, production audit trails, distributed rate limiting and drift monitoring remain incomplete. Live PostgreSQL migration/ingestion and live Snwolley connectivity must be verified in the target environment with a running database and a fresh API key.

8. Local verification status

The source was compiled and the automated suite discovered 20 tests: 14 passed and 6 PostgreSQL-dependent tests were skipped because no PostgreSQL server was available in the isolated verification environment. The model training and inference path ran successfully. Snwolley request construction and report generation were tested with mocked provider responses; a live provider request has not been made. A full Next.js production build remains to be run on a machine with npm dependencies installed.